

BUSINESS ANALYTICS

Group Project Announcement

Real-Time Payment Fraud Detection for an E-Commerce Platform

An End-to-End Analytics Project: From Raw Data to a Monitored, Deployed Model

Course	Business Analytics
Project Type	Group Project (4–5 students per team)
Domain	E-Commerce — Payment Risk & Trust
Deployment Target	Cloud free tier (e.g., Render, Hugging Face Spaces, Streamlit Community Cloud)
Coding Requirement	All code must be written by team members (no no-code/AutoML platforms)

1. Project Overview

E-commerce platforms process thousands of transactions per minute, and even a small fraud rate translates into significant financial loss and damaged customer trust. This project asks each team to act as the risk-analytics function of an e-commerce platform and answer a concrete business question:

“Which incoming transactions are likely to be fraudulent, and how should the platform balance blocking fraud against approving legitimate orders without unnecessary friction?”

Teams will build a classification model that flags high-risk transactions in real time, working with a severely imbalanced dataset that mirrors real-world fraud rates (typically well under 5%). The model will be engineered, deployed as a working application, and monitored after deployment — mirroring the full lifecycle of an analytics product in industry.

2. Data Sources

Teams must combine the following two data sources:

- **Kaggle dataset (required base):** “Online Payments Fraud Detection Dataset” — [kaggle.com/datasets/rupakroy/online-payments-fraud-detection-dataset](https://www.kaggle.com/datasets/rupakroy/online-payments-fraud-detection-dataset). This provides transaction-level records and fraud labels.
- **Synthetic data (required extension):** Each team must generate additional contextual data using Python (e.g., the Faker library plus custom business logic) — customer account age,

device/browser fingerprint, shipping vs. billing address mismatch, number of failed payment attempts, IP-to-billing-country distance, time-of-day pattern, etc.

This combination ensures the project is not a simple “download-and-train” exercise: teams must justify the realism of the data they generate and document it rigorously, and must explicitly address the class-imbalance challenge inherent to fraud data.

Data Dictionary Requirement: Every synthetically generated field must be documented in a data dictionary specifying column name, data type, unit, valid range, and the generation logic or business assumption used.

3. Project Structure (8 Modules)

The project is organized into eight modules, suitable for an 8-week timeline (adjustable to fit the course schedule).

Module	Focus Area	Key Tasks
1	Business Understanding & Data Generation	- Define the business problem and key KPIs (fraud rate, false-positive rate, financial loss avoided) - Generate supplementary contextual/behavioral data in Python - Produce a complete data dictionary
2	Exploratory Data Analysis (EDA)	- Analyze distributions, outliers, and correlations - Quantify class imbalance and compare fraud rates across transaction types, channels, and time windows - Visualize fraud patterns (e.g., transaction amount vs. fraud likelihood, geographic mismatch patterns)
3	Data Cleaning	- Handle missing values, duplicates, and inconsistent categories - Treat outliers (e.g., implausible transaction amounts, invalid timestamps) - Document all cleaning decisions with before/after comparisons
4	Feature Engineering	- Construct risk-signal features (velocity of transactions per account, address mismatch flag, new-device flag, time-since-last-purchase) - Encode categorical variables and scale numerical features - Address severe class imbalance (e.g., SMOTE, undersampling, class weights) and perform feature selection
5	Model Development	Train and compare classifiers: Logistic Regression, Random Forest, XGBoost / LightGBM for fraud prediction

Module	Focus Area	Key Tasks
		- Evaluate with metrics suited to imbalanced data: AUC-PR, Precision/Recall, F1, and a cost-based metric (cost of missed fraud vs. cost of false alarms) - Select an operating threshold that reflects the business trade-off between fraud loss and customer friction
6	Model Deployment	- Package the model as an API (FastAPI or Flask) that scores a transaction in real time - Build a demo interface (Streamlit) simulating a fraud-analyst review queue - Deploy to a cloud free tier (Render, Hugging Face Spaces, or equivalent)
7	Model Monitoring	- Track data drift by comparing incoming transaction distributions to training data - Monitor performance metrics over time, including precision/recall on a rolling window - Build a simple monitoring dashboard (e.g., using the open-source Evidently library) and define retraining triggers
8	Final Report & Presentation	- Translate findings into risk-policy recommendations (e.g., which signals should trigger manual review vs. auto-block) - Discuss limitations and future improvements - Present to the class with a live or recorded demo

4. Deliverables

Deliverable	Format
Source code (notebooks/scripts)	GitHub repository
Data dictionary	Excel or Markdown file
Deployed API + review-queue demo	Live link (Render / Hugging Face Spaces / Streamlit Cloud)
Monitoring dashboard or report	Notebook or Evidently report
Final report and slides	PDF and PPTX

5. Suggested Team Roles (4–5 members)

Role	Responsibilities
Data Engineer	Synthetic data generation, data dictionary, data cleaning

Role		Responsibilities
Data Analyst		Exploratory data analysis and visualization of fraud patterns
ML Engineer Modeling	—	Feature engineering and classification model development
ML Engineer Evaluation	—	Imbalanced-data handling, threshold selection, cost-based evaluation
MLOps Lead		Deployment and monitoring (may be combined with another role for 4-person teams)

6. Evaluation Focus

Grading will emphasize the rigor and clarity of each stage, rather than model accuracy alone:

- Quality and realism of synthetic data, supported by a complete data dictionary
- Depth of EDA and clarity of insights communicated through visualization
- Soundness of cleaning and feature engineering decisions, with clear documentation
- Appropriate handling of class imbalance and choice of evaluation metrics for the business problem
- A working, accessible deployment (live link required)
- A functioning monitoring component with defined drift/performance triggers
- Originality of code — all code must be authored by team members

7. Submission Logistics

Instructors should insert specific dates for each milestone below before distributing this announcement.

1. Team formation and topic confirmation — [insert date]
2. Module 1–3 checkpoint (data + EDA + cleaning) — [insert date]
3. Module 4–5 checkpoint (features + models) — [insert date]
4. Module 6–7 checkpoint (deployment + monitoring) — [insert date]
5. Final submission and presentation — [insert date]

Questions about scope, data sources, or deployment options should be directed to the course instructor.